

CloudShare

Design and Deployment of a Highly Available, Multi-Tier Cloud-Native Application using
the AWS Well-Architected Framework

Case Study Report

Submitted by

Shivam Kumar

Roll Number: 2400290120238

Github Repository

<https://github.com/shivam-attri-85/CloudShare>

Project tagline: Simple. Fast. Private. Your Files. Your Cloud.

AWS Region:US East (N.Virginia) -us-east-1

1. Executive Summary

CloudShare is a lightweight personal cloud file-sharing application designed to provide a simple alternative to advertisement-heavy or overly complex file-transfer services. The implemented prototype exposes a browser-based interface for uploading files, tracking upload progress, viewing stored files, downloading files, and deleting files. The application is built around a Python Flask backend with a responsive HTML/CSS/JavaScript frontend and is deployed on an Ubuntu-based Amazon EC2 instance.

The AWS environment captured in the provided evidence includes an EC2 instance named **CloudShare_API** in the **us-east-1** region, a default VPC, a security group permitting SSH, HTTP and TCP/5000 traffic for the demonstrated deployment, and Amazon S3 buckets used as cloud storage resources. The current runnable prototype stores application uploads in the EC2 filesystem; the S3 resources are documented as the cloud-storage layer available for the next iteration rather than claimed as the current application data path.

Key outcome: the end-to-end prototype was validated with real upload, file-list and download requests against the deployed Flask service. A systemd service was also prepared so the application can be operated independently of an interactive SSH shell when the EC2 instance is running.

2. Abstract

Cloud file sharing requires three practical qualities: accessible storage, a simple user experience and reliable server-side operation. CloudShare addresses these needs through a minimal web application running on AWS. Users can select a file or drag it into the browser, observe upload progress, inspect available files, download selected content and remove files from the server. The backend exposes REST-style endpoints for health checking, upload, listing, download and deletion.

The case study applies the AWS Well-Architected Framework as a design lens. The implementation emphasizes operational simplicity, least-exposure networking, separation of application and storage concerns, monitoring of the EC2 environment, and the ability to evolve toward a multi-instance, multi-Availability-Zone architecture. Because the present evidence shows a single production-like EC2 instance, this report distinguishes between **implemented functionality** and **target architecture/future hardening**.

Keywords: AWS EC2, Amazon S3, Flask, Cloud-Native Application, File Sharing, Well-Architected Framework, Linux, Systemd.

3. Problem Statement and Motivation

Users frequently need to move files between a laptop and a phone or between multiple personal devices. Common web file-transfer services can add advertisements, require extra accounts, or introduce unnecessary workflow steps. Traditional local-only transfer methods also depend on both devices being online at the same time. CloudShare was conceived as a minimal personal service where the user interacts with a single web interface while a remote EC2 server performs the file operations.

The main problem is therefore to build a small but demonstrable cloud-native file-sharing workload that can be deployed remotely, accessed through a browser, and operated with clear AWS infrastructure evidence. The assignment additionally requires reasoning about availability, security, scalability, cost and operational discipline under the Well-Architected Framework.

4. Objectives

- Provide a browser-based personal file-sharing interface.
- Implement file upload with drag-and-drop interaction and visible progress.
- List uploaded files and allow users to download them on demand.
- Provide file deletion through the application interface.
- Expose a health endpoint for basic service validation and monitoring.

- Deploy the application on AWS EC2 and verify network accessibility.
- Use Amazon S3 as the planned durable object-storage layer for evolution of the prototype.
- Keep the deployment simple enough to operate within a student/free-tier learning environment.
- Document the solution using AWS Well-Architected principles and evidence from the actual AWS Console.

5. Scope of the Implemented Solution

Implemented: Flask backend, browser UI, upload API, file listing API, file download API, delete API, drag-and-drop upload area, upload progress display, EC2 deployment, Security Group configuration, Ubuntu virtual environment, and systemd service preparation.

Present limits: the demonstrated application uses EC2 local storage for active uploads; no load balancer or second EC2 instance is evidenced in the submitted screenshots; authentication, HTTPS, per-user authorization and S3-backed application storage are not demonstrated as part of the current running path. These are treated as planned production enhancements rather than retroactively claimed features.

Out of scope for the current prototype: enterprise identity management, malware scanning, encrypted file-at-rest policy design, multipart/resumable transfer, full observability stack, automated infrastructure-as-code, and production-grade disaster recovery.

6. Technology Stack

Layer	Technology	Role
Frontend	HTML5, CSS3, JavaScript	Browser UI, drag/drop, progress bar, file actions
Backend	Python 3.12, Flask	HTTP routes and server-side file operations
API support	Flask-CORS	Cross-origin request support where required
Runtime	Ubuntu 24.04 LTS	EC2 operating system
Compute	Amazon EC2 (t3.micro evidenced)	Application hosting
Storage	EC2 filesystem + Amazon S3 resources	Current local upload path + planned durable object storage
Networking	Amazon VPC, subnet, Security Group	Network isolation and inbound controls
Operations	systemd	Background service and automatic restart
Source control	GitHub Public Repository	Versioning, source and documentation

Repository: <https://github.com/shivam-attri-85/CloudShare>

7. System Architecture

The target cloud-native architecture places end users on the public internet in front of a web/API tier hosted on EC2. Amazon S3 is the intended durable object-storage destination, while security groups limit network access to required ports. For high availability, the reference design can be extended to two EC2 instances across different Availability Zones behind an Application Load Balancer. This multi-instance arrangement is shown in the accompanying architecture figure as the desired scalable design, while the evidence-backed current deployment remains a single EC2 instance.

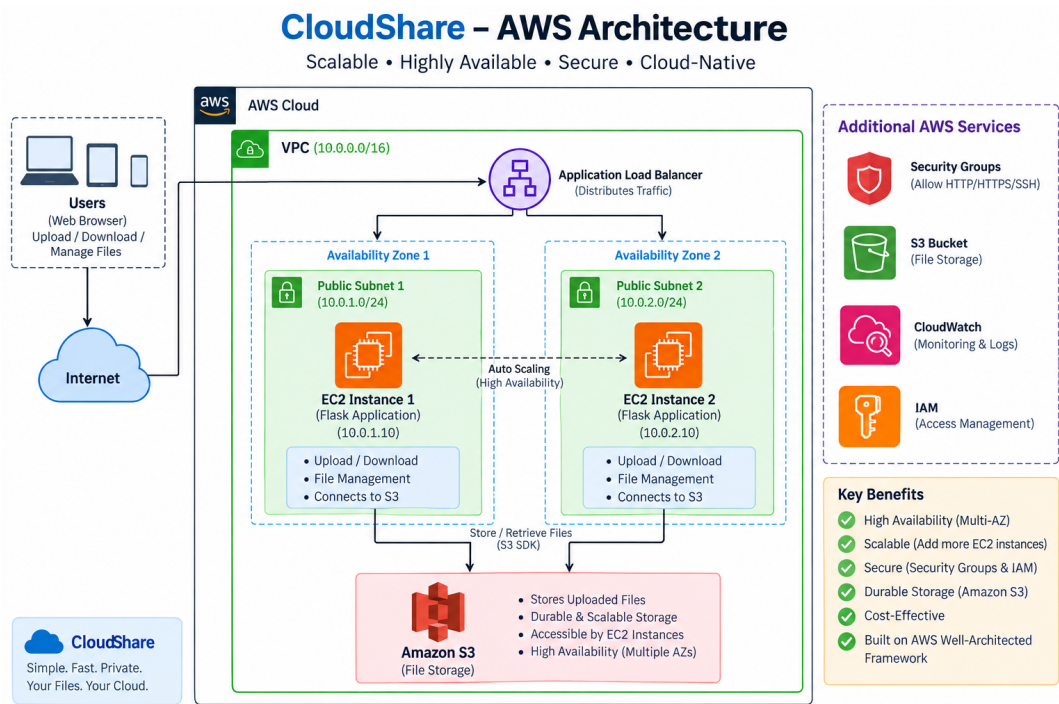


Figure 1. Reference target architecture for CloudShare. The diagram represents the intended high-availability extension; the currently evidenced deployment is a single EC2 instance plus S3 resources.

7.1 Request and Data Flow

- Browser sends a request to the CloudShare web endpoint.

- Flask serves the HTML/CSS/JavaScript interface and handles file API requests.
- For upload, the browser sends multipart form data to **POST /upload**.
- The backend writes the uploaded file into the configured uploads directory in the current prototype.
- **GET /files** returns the available filenames to the browser.
- **GET /download/<filename>** streams a selected file back to the user.
- **DELETE /delete/<filename>** removes an uploaded file after confirmation.
- **GET /health** returns a basic healthy status used for smoke testing.

8. AWS Infrastructure and Deployment Evidence

The submitted screenshots document an AWS deployment in US East (N. Virginia), us-east-1. The evidence includes the EC2 instance, networking attributes, Security Group rules, EBS volume configuration, Amazon S3 buckets, bucket properties, and an application UI served from the EC2 public address.

8.1 Amazon EC2

The EC2 console shows an instance named **CloudShare_API**, type **t3.micro**, in Availability Zone **us-east-1c**. The instance is shown as Running with 3/3 status checks passed. The recorded public IPv4 address is 54.221.65.189 and the private address is 172.31.28.162. The root disk is an 8 GiB EBS volume.

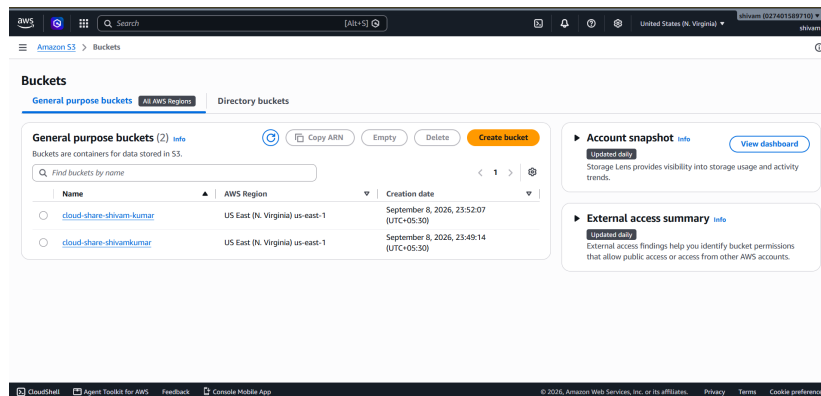


Figure 2. EC2 Instances console showing CloudShare_API running and healthy.

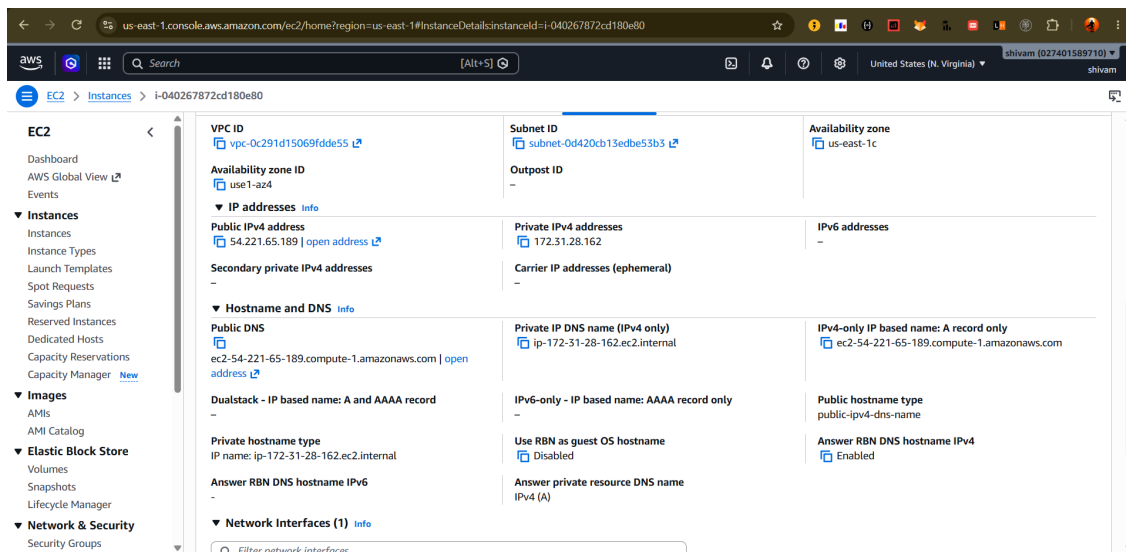


Figure 3. EC2 instance networking details, including VPC, subnet, Availability Zone and IP addresses.

8.2 Security Group and Network Configuration

The Security Group named **CloudShare-API-SG** is visible in the launch configuration. The demonstrated inbound rules allow SSH on port 22 from a specific /32 source, HTTP on port 80, and TCP port 5000 from 0.0.0.0/0 for the Flask development endpoint. For a production deployment, exposing the Flask port publicly should be avoided; traffic should terminate at a reverse proxy or load balancer and the application instances should be reachable only from the appropriate security-group source.

☒ Create security group
 ☐ Select existing security group

Security group name - *required*

CloudShare-API-SG

This security group will be added to all network interfaces. The name can't be edited after the security group is created. Max length is 255 characters. Valid characters: a-z, A-Z, 0-9, spaces, and ._-:/()#,@[]+=&:;!\$*

Description - *required* | Info

Security group for CloudShare API server

Inbound Security Group Rules

- ▶ Security group rule 1 (TCP, 22, 49.43.171.91/32) Remove
- ▶ Security group rule 2 (TCP, 80, 0.0.0.0/0) Remove
- ▶ Security group rule 3 (TCP, 5000, 0.0.0.0/0) Remove

Add security group rule

Figure 4. Security Group configuration used during the EC2 launch.

8.3 Storage and S3 Resources

The S3 console shows two general-purpose buckets created in us-east-1. One bucket is named **cloud-share-shivam-kumar**. Its object listing contains project artifacts such as app.py, backend/, frontend/, requirements.txt, static/, templates/, test-upload.txt and uploads/. The screenshots also show bucket properties with versioning disabled and a separate static/ prefix containing style.css. These objects demonstrate the presence and use of S3 for storing project resources/test artifacts.

Name	Type	Last modified	Size	Storage class
app.py	py	September 9, 2026, 00:02:26 (UTC+05:30)	458.0 B	Standard
backend/	Folder	-	-	-
frontend/	Folder	-	-	-
requirements.txt	txt	September 9, 2026, 00:02:27 (UTC+05:30)	0 B	Standard
static/	Folder	-	-	-
templates/	Folder	-	-	-
test-upload.txt	txt	September 9, 2026, 00:02:28 (UTC+05:30)	38.0 B	Standard
uploads/	Folder	-	-	-

Figure 5. Amazon S3 bucket object listing for cloud-share-shivam-kumar.

cloud-share-shivam-kumar Info

Objects | Metadata | **Properties** | Permissions | Metrics | Management | File systems | Access Points

Bucket overview

AWS Region: US East (N. Virginia) us-east-1 | Amazon Resource Name (ARN): arnaws:s3::cloud-share-shivam-kumar | Creation date: September 8, 2026, 23:52:07 (UTC+05:30)

Bucket Versioning Edit

Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, you can easily recover from both unintended user actions and application failures. [Learn more](#)

Bucket Versioning: Disabled

Multi-factor authentication (MFA) delete

An additional layer of security that requires multi-factor authentication for changing Bucket Versioning settings and permanently deleting object versions. To modify MFA delete settings, use the AWS CLI, AWS SDK, or the Amazon S3 REST API. [Learn more](#)

MFA delete: Disabled

Bucket ABAC Edit

Attribute-based access control (ABAC) is an authorization strategy that defines permissions based on attributes. With ABAC, you can attach tags to your general purpose buckets and AWS Identity and Access Management (IAM)

Figure 6. S3 bucket properties and versioning state shown in the AWS Console.

9. Application Implementation

The implemented application separates presentation from API logic. **app.py** creates the Flask application, enables CORS, registers the backend blueprint, serves the main template, and exposes the health endpoint. **backend/routes.py** manages upload, listing, download and delete operations against the uploads directory. The frontend uses JavaScript XMLHttpRequest for upload progress and fetch for file listing and delete operations.

9.1 User Interface and File Workflow

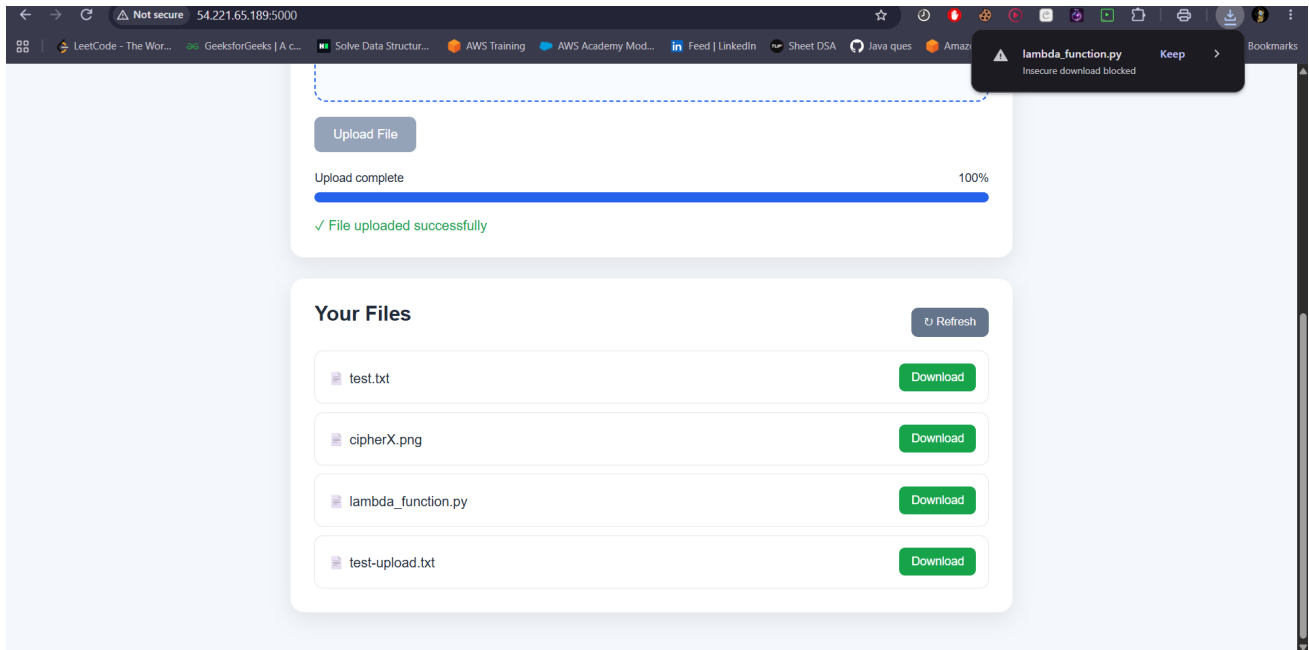


Figure 7. CloudShare web interface showing upload area, upload completion state and the current file list.

The final browser UI supports a simple workflow: choose a file or drag it onto the drop zone, upload it, observe progress, then download files from the list. The interface is intentionally minimal and responsive for use on both desktop and mobile-sized screens.

9.2 API Endpoints

Method	Endpoint	Function
GET	/	Render CloudShare UI
GET	/health	Return healthy service status
POST	/upload	Accept multipart file upload
GET	/files	Return filenames in uploads directory
GET	/download/<filename>	Download selected file
DELETE	/delete/<filename>	Delete selected file

10. Deployment and Testing

The deployment was validated in stages: environment preparation, service startup, health validation, upload, file listing and download. During troubleshooting, intermittent network connection timeouts were observed, followed by successful requests once the Flask process was available. The final successful tests confirmed that files could be transferred through the public EC2 endpoint.

Test Case	Procedure	Expected Result	Observed Result
TC-01	Open /health	JSON healthy status	Passed
TC-02	POST test-upload.txt to /upload	Success response with filename	Passed
TC-03	GET /files	Uploaded filenames returned	Passed
TC-04	GET /download/test-upload.txt	File downloaded	Passed
TC-05	Open / in browser	CloudShare UI displayed	Passed
TC-06	Use drag/drop + progress UI	Visible upload interaction	Implemented / available
TC-07	Delete route from UI	File removed after confirmation	Implemented route; UI integration prepared

Example smoke tests

```
curl http://YOUR_EC2_IP:5000/health
```

```
curl -X POST -F "file=@test-upload.txt" http://YOUR_EC2_IP:5000/upload
```

```
curl http://YOUR_EC2_IP:5000/files
```

```
curl -O http://YOUR_EC2_IP:5000/download/test-upload.txt
```

11. AWS Well-Architected Framework Assessment

The following assessment uses the six AWS Well-Architected pillars as an evaluation framework. The report separates the current prototype state from the target production direction.

11.1 Operational Excellence

The application has a simple deployment workflow, a health endpoint, source code hosted in GitHub, and a systemd unit prepared for automated restart. The current operational gap is limited centralized logging/alerting and limited deployment automation. Next steps are structured logs, CloudWatch monitoring, automated deployment and documented runbooks.

11.2 Security

A dedicated Security Group is used, and SSH access is restricted in the demonstrated launch configuration. The prototype should be hardened by removing public port 5000 exposure, enforcing HTTPS, validating filenames, applying file-size/type limits, using IAM roles instead of long-lived keys, and adding authentication and authorization before broader use.

11.3 Reliability

The EC2 status checks show a healthy instance and systemd provides process restart behavior. However, a single EC2 instance is not a true highly available application tier. The target architecture should use at least two instances in separate Availability Zones behind an Application Load Balancer, with durable S3 storage and health-based routing.

11.4 Performance Efficiency

The application uses a lightweight Flask stack and asynchronous browser-side progress updates. For larger files and higher concurrency, the architecture should offload object transfer to S3 using pre-signed URLs, use a production WSGI server, and introduce caching or multipart uploads where appropriate.

11.5 Cost Optimization

The demonstrated deployment uses a small t3.micro instance and simple storage, suitable for learning and low traffic. Cost control should include stopping unused EC2 resources, lifecycle policies for temporary objects, right-sizing, and avoiding unnecessary public infrastructure.

11.6 Sustainability

The prototype is intentionally small and uses a low-capacity instance. A durable object-storage architecture can reduce the need for permanently attached local capacity, while lifecycle rules and right-sizing can reduce unnecessary resource consumption.

12. High Availability and Multi-Tier Evolution

The assignment title calls for a highly available, multi-tier cloud-native application. The current evidence is the foundation for that architecture, not the final state. The next implementation step is to remove application file persistence from the individual EC2 host and make S3 the authoritative storage layer. Then two EC2 application instances can operate independently behind an Application Load Balancer. Because both instances become stateless with respect to uploaded objects, either instance can serve a subsequent request.

Tier	Current evidence	Target architecture
Presentation	Browser UI served by Flask	ALB + HTTPS entry point
Application	One Flask EC2 instance	2+ EC2 instances across AZs
Storage	Local uploads directory	Amazon S3 bucket as durable object store
Health	/health endpoint + EC2 checks	ALB health checks + CloudWatch
Security	Security Group	ALB-to-EC2 SG chain + IAM role + HTTPS

Tier	Current evidence	Target architecture
Operations	systemd + GitHub	Systemd/ASG or containerized deployment + monitoring

13. Limitations

- Single evidenced EC2 application node; automatic failover between nodes is not yet demonstrated.
- Flask development server is suitable for testing but should not be used as the final public production server.
- TCP port 5000 is publicly exposed in the demonstrated Security Group and should be replaced by HTTPS through a reverse proxy or load balancer.
- Current upload persistence is on the EC2 host filesystem, so instance replacement would threaten file persistence unless storage is moved to S3 or shared storage.
- No user authentication or per-user authorization is demonstrated in the submitted evidence.
- S3 versioning is shown as disabled in the captured bucket properties; enabling it may be appropriate for recovery requirements, with lifecycle controls for cost.

14. Future Enhancements

- Two-EC2 deployment across separate Availability Zones behind an Application Load Balancer.
- Amazon S3-backed file uploads using boto3 and pre-signed URLs.
- Application Load Balancer health checks and Auto Scaling.
- HTTPS using AWS Certificate Manager with a domain name.
- User login, session management and per-user access control.
- Secure filename handling, size/type restrictions, rate limiting and malware scanning.
- CloudWatch metrics/logs and alarms.
- Infrastructure as Code using Terraform or CloudFormation.
- Shareable temporary links and QR codes.
- Automated CI/CD from GitHub.

15. Conclusion

CloudShare successfully demonstrates a working cloud-hosted file-sharing workload with a real web interface and operational AWS infrastructure. The implementation proves the core file lifecycle - upload, list, download and application health - and captures the practical steps required to deploy a Flask service on EC2. The AWS evidence also establishes the presence of VPC networking, Security Group controls, EC2 compute and Amazon S3 storage resources in us-east-1.

From a Well-Architected perspective, the project is a strong prototype foundation but not yet a fully highly available production system. The most valuable architectural improvement is to make the application tier stateless and move uploaded objects to S3, followed by deploying two EC2 instances behind an Application Load Balancer across Availability Zones. This evolution directly improves reliability while preserving the simple CloudShare user experience and keeping the solution aligned with cost-conscious student deployment goals.

16. Submission Checklist

Requirement	Status / Notes
Case Study Report (PDF)	This report
Source Code / Application Files	GitHub repository: https://github.com/shivam-attri-85/CloudShare
AWS Architecture Diagram	Included as target architecture figure; update with final deployed topology when second EC2/
Infrastructure / Deployment Files	Systemd service documented; Terraform/CloudFormation not evidenced in current submission
Deployment Screenshots / Evidence	AWS and application screenshots included